

untangle::executor

Generated by Doxygen 1.18.0

1	executor	1
2	Namespace Index	3
2.1	Namespace List	3
3	Class Index	5
3.1	Class List	5
4	File Index	7
4.1	File List	7
5	Namespace Documentation	9
5.1	untangle Namespace Reference	9
5.1.1	Detailed Description	9
6	Class Documentation	11
6.1	untangle::executor Class Reference	11
6.1.1	Detailed Description	11
6.1.2	Member Typedef Documentation	12
6.1.2.1	task_t	12
6.1.3	Constructor & Destructor Documentation	12
6.1.3.1	executor()	12
6.1.3.2	~executor()	12
6.1.4	Member Function Documentation	13
6.1.4.1	submit()	13
7	File Documentation	15
7.1	executor.hpp	15
	Index	19

Chapter 1

executor

c++ thread pool executor based on async execution

The interface is one header that exposes a pool of workers, each one an **async** execution in continuous mode, fed by a shared queue of tasks.

This is the prototype, imported. It was written in `async/prototypes/` to answer whether execution is a usable building block for a pool — it is — and it is being hardened here. The behaviour below is what the tests state today, not a settled interface.

requirements

C++23, and the `async` submodule:

```
git submodule update --init --recursive
```

`async` carries its own actuator submodule, which is why the clone has to be recursive. Both are header-only; nothing is built or installed.

The headers report warnings with `std::println`, so they need a standard library that provides `<print>` — GCC 14 or newer.

example

```
#include <executor.hpp>

int main() {
    untangle::executor pool(4);

    for (int i = 0; i < 100; ++i) {
        pool.submit([i] { work(i); });
    }

    return 0; // ~executor finishes what was submitted, then stops the workers
}
```

how work is placed

N executions run in continuous mode behind a shared `std::deque` of tasks.

Nothing overtakes. A task goes straight to a free worker only when the shared queue is empty; otherwise it joins the back of the queue. Finding a free worker does not let a task jump a queue that already has work in it.

Workers pull rather than being pushed to. When one reports its queue finished — `on_finished`, raised by execution the moment its list empties — it takes the front of the shared queue. That is what spreads the work, with no scheduling logic of its own: 400 tasks over 4 workers come out 100/100/100/100.

A worker is asked, not tracked. `execution::is_busy()` is what `submit()` reads to find a free worker and what the destructor reads to decide the pool is idle. A tally kept here would be the pool's belief about its workers; this is the workers' own answer, and it cannot drift.

lifetime and failure

The destructor finishes what was submitted. It refuses further work, waits for the queue to empty and for every worker to go idle, and only then stops them. A task submitted before the pool goes out of scope has run by the time it does.

`submit()` says whether the task was taken. It returns false once the pool is shutting down, which a task submitting more work from inside the pool can see.

A task that throws does not take the worker with it. `execution` catches it, reports it on `stderr`, and the worker carries on with the next task. Nothing reaches the submitter — a task that has something to report carries its own channel, because a continuous worker does not collect results.

building the tests

```
cmake -S test -B test/build
cmake --build test/build
ctest --test-dir test/build
```

`test/CMakeLists.txt` fetches googletest at configure time, so the first configure needs network access.

sanitizers

`EXECUTOR_SANITIZE` builds the tests under a sanitizer. It is off by default, because a sanitized build is several times slower and ThreadSanitizer does not ship for every toolchain.

```
cmake -S test -B test/build-asan -DEXECUTOR_SANITIZE=address
cmake --build test/build-asan
ctest --test-dir test/build-asan
```

Accepted values are `address`, `thread`, `undefined`, or `empty`. Anything else is refused at configure time rather than passed through to the compiler.

Use a separate build directory per sanitizer. `address` and `thread` instrument the same accesses in incompatible ways and cannot be combined, so one build directory is one sanitizer.

the reference

`tools/make_doc.sh` builds `doc/refman.pdf` from the header and this README. It needs `doxygen`, `tectonic` and `python3`, and it fails on any `doxygen` warning that is not an `obsolete-tag` notice, so an undocumented member or a broken `\ref` does not reach the reference.

Chapter 2

Namespace Index

2.1 Namespace List

Here is a list of all documented namespaces with brief descriptions:

untangle	A thread pool executor built out of <code>untangle::async::execution</code> objects	9
--------------------------	---	-------------------

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

untangle::executor	Runs tasks on a fixed pool of <code>untangle::async::execution</code> workers	11
------------------------------------	---	--------------------

Chapter 4

File Index

4.1 File List

Here is a list of all documented files with brief descriptions:

executor.hpp	15
--	----

Chapter 5

Namespace Documentation

5.1 untangle Namespace Reference

A thread pool executor built out of `untangle::async::execution` objects.

Classes

- class `executor`
Runs tasks on a fixed pool of `untangle::async::execution` workers.

5.1.1 Detailed Description

A thread pool executor built out of `untangle::async::execution` objects.

Imported from the `async` repo's `prototypes/executor.hpp`, unchanged apart from its namespace and this comment. The hardening the prototype's README asks for has not been done yet.

The pool holds `N` executions in continuous mode and a shared queue of tasks. A task is handed straight to a free worker only when the shared queue is empty; otherwise it goes to the back of the queue, and a worker takes the front of the queue as it frees up. Nothing overtakes: finding a free worker does not let a task jump a queue that already has work in it.

Chapter 6

Class Documentation

6.1 `untangle::executor` Class Reference

Runs tasks on a fixed pool of `untangle::async::execution` workers.

```
#include <executor.hpp>
```

Public Types

- using `task_t` = `std::function<void(void)>`

Public Member Functions

- `executor` (`std::size_t worker_count`)
Starts `worker_count` executions in continuous mode.
- `~executor` ()
Finishes what has been submitted, then stops every worker.
- `bool submit` (`task_t` task)
Queues a task, or hands it to a worker that has nothing to do.
- `std::size_t pending` () `const`
How many tasks are waiting for a worker.

6.1.1 Detailed Description

Runs tasks on a fixed pool of `untangle::async::execution` workers.

Remarks

Tasks are `std::function<void(void)>`. An execution's action type is fixed by its template argument, so one pool runs one signature; a task that needs to return something carries its own channel, because a continuous worker does not collect results.

6.1.2 Member Typedef Documentation

6.1.2.1 task_t

```
using untangle::executor::task_t = std::function<void(void)>
```

The work a pool runs. One pool runs one signature, because an execution's action type is fixed by its template argument.

6.1.3 Constructor & Destructor Documentation

6.1.3.1 executor()

```
untangle::executor::executor (
    std::size_t worker_count) [inline], [explicit]
```

Starts worker_count executions in continuous mode.

Parameters

worker_count	- How many workers to run. Must not be zero.
--------------	--

Exceptions

std::invalid_argument	- worker_count is zero. Such a pool would take work and never run it, so it is refused before any worker starts rather than left to hang in the destructor.
-----------------------	---

Remarks

std::thread::hardware_concurrency() returns 0 when it cannot tell how many cores there are, and a caller passing that straight through is the likely way in. Choosing a default then is the caller's call, not the pool's.

6.1.3.2 ~executor()

```
untangle::executor::~~executor () [inline]
```

Finishes what has been submitted, then stops every worker.

Both waits are unbounded - a pool that abandoned a running task would be worse than one that waits for it - and both report on stderr rather than stalling in silence, naming the workers they are still waiting on.

Remarks

Every worker has left its thread before the first execution is destroyed. ~execution() waits only for its own worker, so destroying them one at a time would free the first while another's thread was still reading it through nothing_running(); the poll waits for all of them. It is this pool's own poll, so a second pool running alongside it is not waited for.

6.1.4 Member Function Documentation

6.1.4.1 submit()

```
bool untangle::executor::submit (  
    task\_t task) [inline]
```

Queues a task, or hands it to a worker that has nothing to do.

Returns

true - the task was accepted. false - the pool is shutting down and refused it.

The documentation for this class was generated from the following file:

- executor.hpp

Chapter 7

File Documentation

7.1 executor.hpp

```
00001 // Copyright (c) 2026 Nicolae Popescu. MIT License.
00002
00014 #pragma once
00015
00016 #include <algorithm>
00017 #include <async.hpp>
00018 #include <chrono>
00019 #include <condition_variable>
00020 #include <cstdint>
00021 #include <deque>
00022 #include <functional>
00023 #include <memory>
00024 #include <mutex>
00025 #include <print>
00026 #include <stdexcept>
00027 #include <string>
00028 #include <thread>
00029 #include <utility>
00030 #include <vector>
00031
00032 namespace untangle {
00033
00041 class executor {
00042 public:
00045     using task_t = std::function<void(void)>;
00046
00059     explicit executor(std::size_t worker_count) {
00060         if (worker_count == 0) {
00061             throw std::invalid_argument("executor: worker_count must not be zero");
00062         }
00063
00064         workers_.reserve(worker_count);
00065
00066         for (std::size_t index = 0; index < worker_count; ++index) {
00067             auto next = worker_execution::create_instance("pool_worker_" + std::to_string(index));
00068
00069             // How a worker learns there is more waiting for it. is_busy() answers whether a worker has
00070             // room; this is what tells the pool the moment one frees up, without anybody polling.
00071             next->on_finished = [this, index] { take_next_task(index); };
00072
00073             // A worker runs in a detached thread and cannot be joined, so asking whether it is still
00074             // running is the only way to wait for one, and the poll answers that for any number of them
00075             // at once. A poll answers for the executions added to it, and this one is added the pool's
00076             // own workers and nothing else, so what the destructor waits for is exactly these. Nothing
00077             // here has to unregister - the record runs both ways, so whichever of the two dies first
00078             // takes itself out of the other.
00079             poll_.add(*next);
00080
00081             workers_.push_back(std::move(next));
00082             workers_.back()->start();
00083         }
00084     }
00085
00098     ~executor() {
00099     {
00100         std::unique_lock<std::mutex> lock(mutex_);
00101         accepting_ = false;
```

```

00102
00103 // Unbounded, and reported rather than bounded. Both waits below keep waiting for as long as
00104 // it takes; what they stop doing is keeping it to themselves.
00105 auto interval = std::chrono::milliseconds(report_first_ms);
00106 auto waited = std::chrono::milliseconds(0);
00107
00108 while (!finished_cv_.wait_for(lock, interval,
00109                               [this] { return pending_.empty() && nothing_running(); })) {
00110     waited += interval;
00111
00112     // The lock is held here, which is what makes both reads consistent with the predicate that
00113     // just failed. is_busy() takes each execution's action_mutex_ under this one, which is the
00114     // same order give_to_worker() uses - no new lock order is introduced by reporting.
00115     std::println(
00116         stderr, "executor: still waiting to finish after {}s - {} queued, {} in a task{}",
00117         waited.count() / 1000, pending_.size(), count_workers(&worker_execution::is_busy),
00118         name_workers(&worker_execution::is_busy));
00119
00120     interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00121 }
00122 }
00123
00124 // Outside the lock: stop() wakes each worker, and a worker waking up runs on_finished, which
00125 // wants this mutex.
00126 for (auto& one : workers_) {
00127     one->stop();
00128 }
00129
00130 // The poll stands in for a join. Until it reports idle, a worker can still be inside
00131 // take_next_task(), reading every execution through nothing_running() and is_busy() - which is
00132 // what makes destroying them here a use-after-free rather than a teardown.
00133 //
00134 // A different question from the finish above, and reported as one: that wait is about workers
00135 // still inside a task, this is about workers whose thread has not left. A worker can be idle
00136 // and still be here.
00137 auto interval = std::chrono::milliseconds(report_first_ms);
00138 auto waited = std::chrono::milliseconds(0);
00139
00140 while (poll_.is_running()) { // time of check
00141     const auto tick = std::chrono::milliseconds(poll_interval_ms);
00142     std::this_thread::sleep_for(tick);
00143     waited += tick;
00144
00145     if (waited >= interval) {
00146         std::println(stderr, "executor: still waiting to stop after {}s - {} not left its thread{}",
00147                     waited.count() / 1000, count_workers(&worker_execution::is_running),
00148                     name_workers(&worker_execution::is_running));
00149
00150         interval = std::min(interval * 2, std::chrono::milliseconds(report_max_ms));
00151     }
00152 }
00153
00154 // time of use: no worker thread is left to reach workers_, so clearing it races nothing.
00155 workers_.clear();
00156 }
00157
00163 bool submit(task_t task) {
00164     std::lock_guard<std::mutex> lock(mutex_);
00165
00166     if (!accepting_) {
00167         return false;
00168     }
00169
00170     // The queue comes first whenever it has anything in it: a task that arrives while others are
00171     // waiting does not get to overtake them just because a worker happens to be free.
00172     if (pending_.empty()) {
00173         for (std::size_t index = 0; index < workers_.size(); ++index) {
00174             if (!workers_[index]->is_busy()) {
00175                 give_to_worker(index, std::move(task));
00176                 return true;
00177             }
00178         }
00179     }
00180
00181     pending_.push_back(std::move(task));
00182     return true;
00183 }
00184
00188 std::size_t pending() const {
00189     std::lock_guard<std::mutex> lock(mutex_);
00190     return pending_.size();
00191 }
00192
00193 private:
00194     using worker_execution = untangle::async::execution<std::function<void(void)>>;
00195
00202     void give_to_worker(std::size_t index, task_t task) {

```

```

00203 // add_action() takes the execution's own action_mutex while this holds mutex_. That is only
00204 // safe in one direction, and it holds: a worker calls on_finished with action_mutex released,
00205 // so it never takes mutex_ while holding action_mutex, and the two never form a cycle.
00206 workers_[index]->add_action(std::move(task));
00207 }
00208
00212 void take_next_task(std::size_t index) {
00213     std::lock_guard<std::mutex> lock(mutex_);
00214
00215     if (!pending_.empty()) {
00216         task_t next = std::move(pending_.front());
00217         pending_.pop_front();
00218         give_to_worker(index, std::move(next));
00219         return;
00220     }
00221
00222     if (nothing_running()) {
00223         finished_cv_.notify_all();
00224     }
00225 }
00226
00233 bool nothing_running() const {
00234     for (const auto& one : workers_) {
00235         if (one->is_busy()) {
00236             return false;
00237         }
00238     }
00239
00240     return true;
00241 }
00242
00248 std::size_t count_workers(bool (worker_execution::*state)() const) const {
00249     std::size_t count = 0;
00250
00251     for (const auto& one : workers_) {
00252         if ((one.get()->*state)()) {
00253             ++count;
00254         }
00255     }
00256
00257     return count;
00258 }
00259
00270 std::string name_workers(bool (worker_execution::*state)() const) const {
00271     std::string names;
00272
00273     for (const auto& one : workers_) {
00274         if ((one.get()->*state)()) {
00275             names += names.empty() ? " : " : ", ";
00276             names += one->name;
00277         }
00278     }
00279
00280     return names;
00281 }
00282
00285 static constexpr int poll_interval_ms = 50;
00286
00289 static constexpr int report_first_ms = 1000;
00290 static constexpr int report_max_ms = 30000;
00291
00292 mutable std::mutex mutex_;
00293 std::condition_variable finished_cv_;
00294
00305 async::execution_poll poll_;
00306
00307 std::deque<task_t> pending_;
00308 std::vector<std::shared_ptr<worker_execution> > workers_;
00309 bool accepting_ = true;
00310 };
00311
00312 } // namespace untangle

```


Index

- ~executor
 - [untangle::executor](#), [12](#)
- executor, [1](#)
 - [untangle::executor](#), [12](#)
- submit
 - [untangle::executor](#), [13](#)
- task_t
 - [untangle::executor](#), [12](#)
- untangle, [9](#)
- untangle::executor, [11](#)
 - [~executor](#), [12](#)
 - [executor](#), [12](#)
 - [submit](#), [13](#)
 - [task_t](#), [12](#)